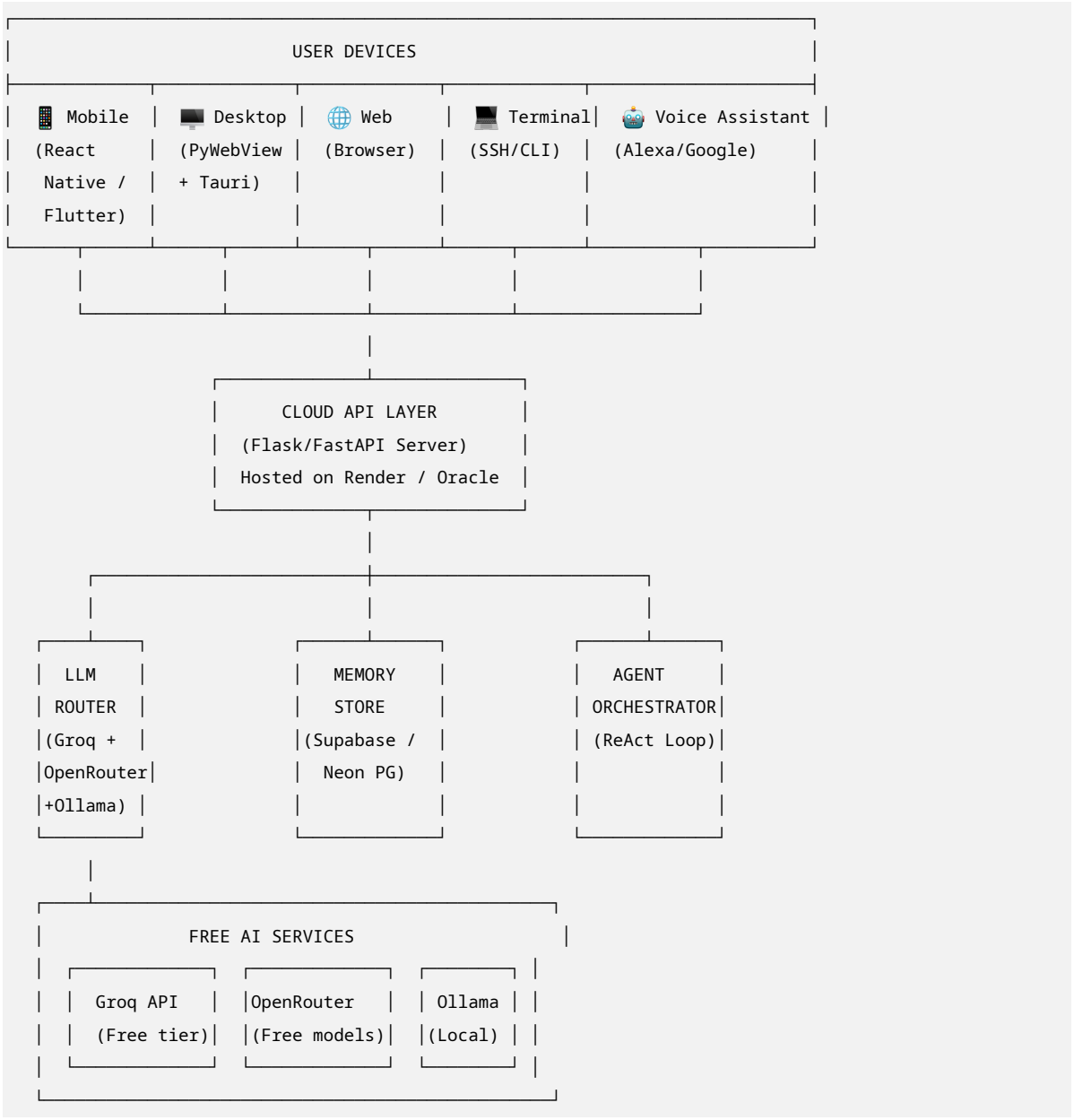
🌐 NEO AI — Complete Deployment Architecture

Mobile App | Desktop App | Web App | Terminal | Cloud | Always Online

> **Goal:** Deploy Neo AI across every platform, keep it always online, and spend $0.

📊 Architecture Overview



---

1 FREE CLOUD HOSTING OPTIONS (Always Online)

🥇 TIER 1: Render (Recommended for Neo)

Feature	Details
Free Tier	750 instance hours/month
Sleep Policy	Sleeps after 15 min idle (30-50s cold start)
Credit Card	❌ Not required
Best For	Flask/FastAPI backend, static sites
Limitations	Cold starts on free tier; paid $7/mo for always-on

Deploy Neo Backend:

```
# 1. Push Neo to GitHub
# 2. Connect repo to Render
# 3. Set build command: pip install -r requirements.txt
# 4. Set start command: gunicorn -w 2 -b 0.0.0.0:$PORT launcher:app
```

🥈 TIER 2: Oracle Cloud Always Free (Most Powerful)

Feature	Details
Free Tier	4 ARM cores + 24GB RAM + 200GB storage
Sleep Policy	❌ Never sleeps —truly always-on
Credit Card	✅ Required for verification
Best For	Full self-hosted stack, Ollama GPU inference
Limitations	ARM architecture (some packages need adjustment)

Why it's perfect for Neo:

- Run Ollama locally on the VM (free local LLM inference)
- Host Flask API + PostgreSQL + Redis on same machine
- 24GB RAM handles multiple concurrent users
- Truly always-on, no cold starts

Setup:

```
# SSH into Oracle VM
ssh ubuntu@your-oracle-ip

# Install dependencies
sudo apt update && sudo apt install -y python3-pip docker.io

# Run Ollama
sudo docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama

# Pull your fallback model
sudo docker exec -it ollama ollama pull qwen2.5:1.5b
```

```
# Clone and run Neo
git clone https://github.com/yourusername/neo-ai.git
cd neo-ai
pip install -r requirements.txt
python launcher.py --server
```

🥉 TIER 3: Google Cloud Run (Serverless)

Feature	Details
Free Tier	2M requests/month + 180K vCPU-seconds
Sleep Policy	Scales to 0 (5-15s cold start)
Credit Card	❌ Not required
Best For	Event-driven, bursty traffic
Limitations	Request timeout limits; stateless only

🏅 TIER 4: Koyeb (Global Edge)

Feature	Details
Free Tier	1 Nano instance (512MB RAM, 0.1 vCPU)
Sleep Policy	❌ No sleep
Credit Card	❌ Not required
Best For	Global edge deployment
Limitations	Very limited resources

🏅 TIER 5: Cloudflare Workers (Edge Functions)

Feature	Details
Free Tier	100K requests/day
Sleep Policy	❌ No sleep —0ms cold start
Credit Card	❌ Not required
Best For	API proxy, lightweight edge logic
Limitations	10ms CPU limit; Python via Pyodide (beta)

Use case for Neo: Deploy a lightweight API gateway that routes to your main backend.

---

2 FREE DATABASE & STORAGE

🗄️ Supabase (Recommended)

Feature	Free Tier
Database	PostgreSQL 500MB
Auth	50K MAU
Storage	1GB
Realtime	✅ WebSocket subscriptions
Vector DB	✅ pgvector included
Sleep	Pauses after 1 week inactivity

Why Supabase for Neo:

- Store chat history, user profiles, agent memory
- Realtime subscriptions for live chat updates
- pgvector for semantic search over Neo's knowledge
- Row Level Security for multi-user Neo

```
# Neo Supabase Integration
from supabase import create_client

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_KEY")

supabase = create_client(SUPABASE_URL, SUPABASE_KEY)

# Save chat message
def save_chat(user_id, message, response, model):
    supabase.table("chats").insert({
        "user_id": user_id,
        "message": message,
        "response": response,
        "model": model,
        "timestamp": datetime.now().isoformat()
    }).execute()

# Get chat history
def get_chat_history(user_id, limit=50):
    return supabase.table("chats")        .select("*")        .eq("user_id", user_id)
     ↪  .order("timestamp", desc=True)        .limit(limit)        .execute().data
```

🗄️ Neon (Serverless Postgres)

Feature	Free Tier
Storage	0.5GB
Compute	100 CU-hours/month
Branching	✅ Git-like DB branches
Sleep	Autosuspend (scales to 0)

Table 7 – continued

Feature	Free Tier
Connection	Serverless driver (no connection limits)

🗄️ Turso (SQLite at Edge)

Feature	Free Tier
Storage	9GB
Databases	500
Reads	1B/month
Sleep	❌ Never
Best For	Edge-deployed SQLite, mobile sync

---

3 FREE NOTIFICATION SERVICES

📲 Push Notifications

Service	Free Tier	Best For
Firebase Cloud Messaging (FCM)	Unlimited	Android + iOS + Web push
OneSignal	Unlimited subscribers	Cross-platform, easy SDK
Expo Push	Free (with Expo)	React Native apps

FCM Setup for Neo:

```
# Notify user when Neo finishes a long task
import firebase_admin
from firebase_admin import messaging

firebase_admin.initialize_app()

def send_push(token, title, body, data=None):
    message = messaging.Message(
        notification=messaging.Notification(title=title, body=body),
        data=data or {},
        token=token,
    )
    messaging.send(message)

# Usage: Notify when agent task completes
send_push(
    user_fcm_token,
    "Neo Agent Complete",
```

```
    "Your research on 'AI in Kenya' is ready!",
    {"chat_id": "123", "type": "agent_complete"}
)
```

📧 Email Notifications

Service	Free Tier	Best For
Resend	100 emails/day	Transactional email
Mailgun	5K emails/month (3 months)	Scale
SendGrid	100 emails/day	Reliability

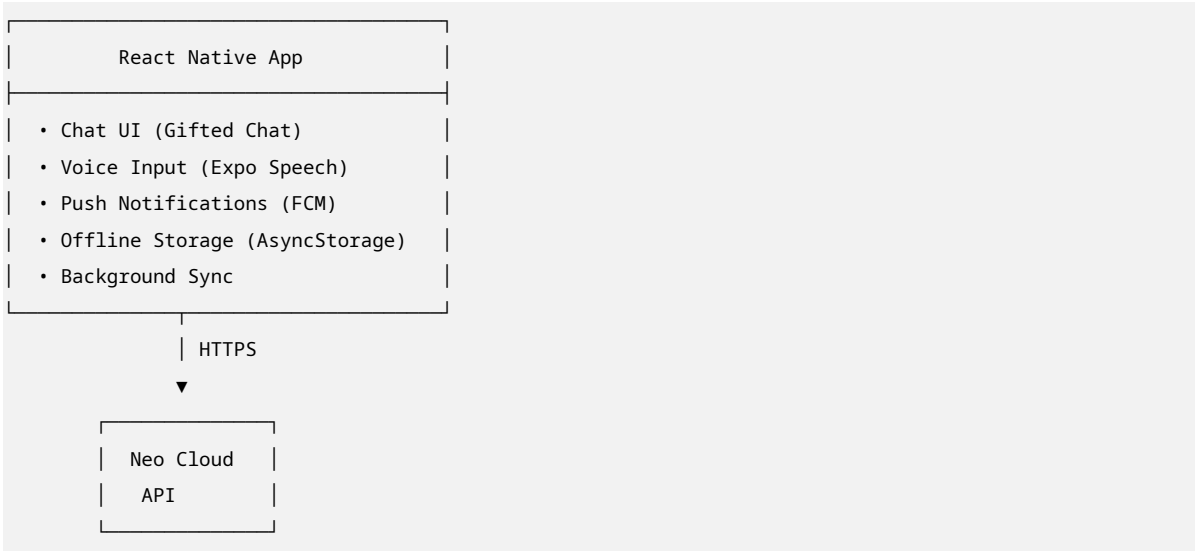
💬 SMS Notifications

Service	Free Tier	Best For
Africa' s Talking	Pay-as-you-go	Africa-focused
Textbelt	1 SMS/day	Global, simple API
AWS SNS	1M requests/month	AWS ecosystem

---

4 MOBILE APP (React Native)

Architecture



Key Libraries

```
npx create-expo-app NeoMobile
cd NeoMobile
```

```
npm install @react-navigation/native @react-navigation/stack
npm install react-native-gifted-chat
npm install @react-native-voice/voice
npm install @react-native-async-storage/async-storage
npm install axios
npm install firebase  # For FCM
```

Neo Mobile Chat Screen

```
// screens/ChatScreen.tsx
import React, { useState, useEffect, useCallback } from 'react';
import { GiftedChat, IMessage, Bubble } from 'react-native-gifted-chat';
import axios from 'axios';
import AsyncStorage from '@react-native-async-storage/async-storage';

const API_URL = 'https://your-neo-api.onrender.com'; // or Oracle IP

export default function ChatScreen() {
  const [messages, setMessages] = useState<IMessage[]>([]);
  const [isTyping, setIsTyping] = useState(false);

  useEffect(() => {
    loadHistory();
  }, []);

  const loadHistory = async () => {
    const history = await AsyncStorage.getItem('neo_chat_history');
    if (history) setMessages(JSON.parse(history));
  };

  const onSend = useCallback(async (newMessages: IMessage[] = []) => {
    const userMsg = newMessages[0];
    setMessages(prev => GiftedChat.append(prev, newMessages));
    setIsTyping(true);

    try {
      const res = await axios.post(`${API_URL}/api/chat`, {
        message: userMsg.text,
        stream: false,
      });

      const neoMsg: IMessage = {
        _id: Date.now(),
        text: res.data.response,
        createdAt: new Date(),
        user: { _id: 2, name: 'Neo', avatar: 'https://neo.ai/avatar.png' },
        ...(res.data.model && { model: res.data.model }),
      };

      setMessages(prev => GiftedChat.append(prev, [neoMsg]));
```

```
      // Save to local storage
      const updated = GiftedChat.append(messages, [...newMessages, neoMsg]);
      await AsyncStorage.setItem('neo_chat_history', JSON.stringify(updated));
    } catch (e) {
      console.error(e);
    } finally {
      setIsTyping(false);
    }
  }, [messages]);

  return (
    <GiftedChat
      messages={messages}
      onSend={onSend}
      user={{ _id: 1 }}
      isTyping={isTyping}
      renderBubble={(props) => (
        <Bubble
          {...props}
          wrapperStyle={{
            right: { backgroundColor: '#10a37f' },
            left: { backgroundColor: '#2d2d2d' },
          }}
        />
      )}
    />
  );
}
```

---

5 DESKTOP APP (Tauri — Better than PyWebView)

Why Tauri over PyWebView?

- **Smaller bundle** (~600KB vs ~50MB)
- **Native performance** (Rust backend)
- **Cross-platform** (Windows, Mac, Linux)
- **Auto-updater** built-in
- **Native notifications**

Setup

```
# Install Tauri prerequisites
npm install -g @tauri-apps/cli

# Create Tauri app
cargo create-tauri-app neo-desktop
cd neo-desktop

# Build
```

```
npm run tauri build
```

Tauri Config (`tauri.conf.json`)

```
{
  "build": {
    "beforeBuildCommand": "npm run build",
    "beforeDevCommand": "npm run dev",
    "devPath": "http://localhost:8000",
    "distDir": "../dist"
  },
  "tauri": {
    "allowlist": {
      "all": false,
      "shell": {
        "all": false,
        "open": true
      },
      "notification": {
        "all": true
      },
      "fs": {
        "all": true,
        "scope": ["$APPDATA/**", "$DOWNLOAD/**"]
      }
    },
    "windows": [
      {
        "title": "Neo AI",
        "width": 1200,
        "height": 800,
        "minWidth": 800,
        "minHeight": 600,
        "center": true,
        "decorations": true,
        "transparent": false
      }
    ]
  }
}
```

---

6 TERMINAL / CLI ACCESS

SSH into Neo Cloud

```
# If hosting on Oracle Cloud
ssh -L 8000:localhost:8000 ubuntu@your-oracle-ip
# Now access Neo at http://localhost:8000 on your local machine
```

```
# Or use Neo CLI directly
ssh ubuntu@your-oracle-ip "cd neo-ai && python -c "from core.local_llm import LocalLLM; llm =
↪  LocalLLM(); print(llm.generate('Hello Neo'))""
```

Telegram Bot Interface

```
# core/telegram_bot.py
import os
import asyncio
from telegram import Update
from telegram.ext import Application, CommandHandler, MessageHandler, filters, ContextTypes

TELEGRAM_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("§$\vcenter{{\hbox{{\includegraphics[height=\ht\strutbox]{{/usr/
    ↪  share/docminer/emoji/1f916.png}}}}}}$§ Neo AI Online. Send me anything!")

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    user_msg = update.message.text
    await update.message.chat.send_action(action="typing")

    # Call Neo backend
    from core.services.neo_brain import get_brain
    brain = get_brain()
    response = await brain.respond(user_msg)

    await update.message.reply_text(response)

app = Application.builder().token(TELEGRAM_TOKEN).build()
app.add_handler(CommandHandler("start", start))
app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))

# Run: python -m core.telegram_bot
```

WhatsApp Bot (via WhatsApp Business API or free alternatives)

```
# Using free WhatsApp Business API via Meta Cloud
# Or use: https://github.com/WhiskeySockets/Baileys (unofficial, free)
```

---

7 VOICE ASSISTANT INTEGRATION

Alexa Skill

```
# lambda_function.py for AWS Lambda (free tier: 1M requests)
import json
import requests

NEO_API = "https://your-neo-api.onrender.com/api/chat"
```

```
def lambda_handler(event, context):
    intent = event['request']['intent']['name']

    if intent == 'AskNeoIntent':
        query = event['request']['intent']['slots']['query']['value']

        # Call Neo API
        res = requests.post(NEO_API, json={"message": query, "stream": False})
        answer = res.json().get('response', 'Sorry, I could not process that.')

        return {
            'version': '1.0',
            'response': {
                'outputSpeech': {
                    'type': 'PlainText',
                    'text': answer[:8000]  # Alexa limit
                }
            }
        }
```

Google Assistant (Actions on Google)

```
// webhook for Dialogflow
const {WebhookClient} = require('dialogflow-fulfillment');
const axios = require('axios');

exports.dialogflowWebhook = async (req, res) => {
  const agent = new WebhookClient({request: req, response: res});
  const query = agent.query;

  const neoRes = await axios.post('https://your-neo-api.onrender.com/api/chat', {
    message: query,
    stream: false
  });

  agent.add(neoRes.data.response);
};
```

---

8 COMPLETE DEPLOYMENT GUIDE

Step 1: Prepare Neo for Cloud

Create `requirements.txt`:

```
flask>=2.0
flask-cors>=4.0
gunicorn>=21.0
python-dotenv>=1.0
requests>=2.31
```

```
aiohttp>=3.9
sentence-transformers>=2.2
faiss-cpu>=1.7
numpy>=1.24
supabase>=2.0
```

Create `Procfile` (for Render/Heroku):

```
web: gunicorn -w 2 -b 0.0.0.0:$PORT launcher:app
```

Create `Dockerfile` (for Oracle/Koyeb):

```
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["gunicorn", "-w", "2", "-b", "0.0.0.0:8000", "launcher:app"]
```

Step 2: Environment Variables

Create `.env`:

```
# LLM Keys
GROQ_API_KEY=your_groq_key
OPENROUTER_API_KEY=your_openrouter_key

# Database
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_KEY=your-anon-key

# Notifications
FIREBASE_PROJECT_ID=your-project
TELEGRAM_BOT_TOKEN=your-telegram-token

# Neo Config
NEO_API_HOST=0.0.0.0
NEO_API_PORT=8000
NEO_DEBUG=false
```

Step 3: Deploy to Render (Easiest)

1. Push Neo to GitHub
2. Go to render.com → "New Web Service"
3. Connect your GitHub repo

4. Set:
   - **Build Command:** `pip install -r requirements.txt`
   - **Start Command:** `gunicorn -w 2 -b 0.0.0.0:$PORT launcher:app`
5. Add environment variables from `.env`
6. Click "Create Web Service"
7. Done! Your API is live at `https://neo-ai.onrender.com`

Step 4: Deploy to Oracle Cloud (Always-On)

1. Sign up at cloud.oracle.com
2. Create Always Free ARM instance (4 OCPU, 24GB RAM)
3. SSH in and run:

```
git clone https://github.com/yourusername/neo-ai.git
cd neo-ai
pip install -r requirements.txt

# Install PM2 for process management
npm install -g pm2

# Start Neo with PM2 (auto-restart, logs)
pm2 start "gunicorn -w 4 -b 0.0.0.0:8000 launcher:app" --name neo-ai
pm2 startup
pm2 save

# Neo is now always online, even after reboot!
```

Step 5: Connect Mobile App

Update `API_URL` in your React Native app:

```
const API_URL = 'https://neo-ai.onrender.com';  // or Oracle IP
```

Build and publish:

```
cd NeoMobile
npx expo build:android   # or :ios
# Or use EAS: npx eas build --platform android
```

Step 6: Set Up Push Notifications

1. Create Firebase project → Add Android/iOS app
2. Download `google-services.json` / `GoogleService-Info.plist`
3. Add to React Native project
4. Store FCM tokens in Supabase:

```
-- Supabase migration
```

```sql
CREATE TABLE user_devices (
    id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id),
    fcm_token TEXT NOT NULL,
    platform TEXT CHECK (platform IN ('android', 'ios', 'web')),
    created_at TIMESTAMP DEFAULT NOW()
);
```

---

9 COST SUMMARY (Monthly)

Component	Service	Cost
Cloud Hosting	Oracle Cloud Always Free	**$0**
Database	Supabase Free	**$0**
LLM Inference	Groq + OpenRouter Free	**$0**
Push Notifications	Firebase FCM	**$0**
Email	Resend (100/day)	**$0**
Domain	Freenom / DuckDNS	**$0**
SSL	Let' s Encrypt	**$0**
Monitoring	UptimeRobot (50 monitors)	**$0**
Total		**$0**

🔟 MONITORING & UPTIME

UptimeRobot (Free)

- Monitor `https://your-neo-api.onrender.com/api/health`
- Get email/SMS alerts if Neo goes down
- 5-minute checks on free tier

Health Check Endpoint

Already built into Neo:

```bash
curl https://your-neo-api.onrender.com/api/health
# → {"status": "ok", "brain": true, "llm": true}
```

Auto-Restart (PM2 on Oracle)

```bash
pm2 monit        # Live dashboard
pm2 logs neo-ai  # View logs
pm2 restart neo-ai  # Manual restart
```

🚀 QUICK START CHECKLIST

- Push Neo to GitHub
- Deploy backend to Render or Oracle Cloud
- Set up Supabase database
- Create React Native mobile app
- Build Tauri desktop app
- Set up FCM push notifications
- Create Telegram bot
- Configure Alexa/Google Assistant skills
- Add UptimeRobot monitoring
- Share Neo with the world! 🌍

📚 Architecture Decision Matrix

If you want···	Use···	Why
Easiest deploy	Render	Git push → live
Always-on, most power	Oracle Cloud	4 cores, 24GB RAM, never sleeps
Global edge	Cloudflare Workers	0ms cold start
Full backend + auth	Supabase	DB + auth + storage + realtime
Pure serverless DB	Neon	Scale-to-zero, branching
Mobile push	Firebase FCM	Free, unlimited
Cross-platform desktop	Tauri	Small, fast, native
Quick mobile app	React Native + Expo	One codebase, iOS + Android

---

Next Step: Pick one cloud provider (I recommend **Oracle Cloud** for always-on + **Render** as backup), deploy your backend, and start building the mobile app. Want me to generate any specific file (Dockerfile, mobile app scaffold, Terraform config, etc.)?

Generated by Kimi.ai